

[Categorías de Industria Estandarizadas para el Screener]

- |— 1. TECH_SOFTWARE (Software, SaaS, IA, Servicios Digitales)
- |— 2. RETAIL_ECOMMERCE (Comercio Minorista, Comercio Electrónico)
- |— 3. HEAVY_INDUSTRY (Manufactura, Industria Pesada, Aerolíneas, Automoción)
- |— 4. FOOD_RESTAURANTS (Restaurantes, Alimentación, Bienes de Consumo Básico)
- |— 5. ENERGY_PHARMA (Petróleo y Gas, Renovables, Farmacéuticas, Biotecnología)
- |— 6. PROFESSIONAL_SERVICES (Consultoría, Servicios Financieros, Inmobiliario)

Tabla 1: **dim_companies** (Datos Maestros Estáticos)

- **ticker** (TEXT, PRIMARY KEY): Identificador único (p.ej. 'AAPL', 'MSFT').
- **company_name** (TEXT): Nombre de la empresa.
- **gics_sector** (TEXT): Sector oficial del proveedor.
- **standardized_industry** (TEXT): Una de las 6 categorías estandarizadas.
- **is_high_growth** (BOOLEAN): Flag calculado dinámicamente por el script de Revenue.

Tabla 2: **fact_financial_statements_raw** (Histórico de Estados Financieros)

Esta tabla almacena las partidas contables puras extraídas, permitiendo recalcular cualquier ratio en el futuro.

- **fact_id** (TEXT, PRIMARY KEY): Combinación de **ticker_year_period** (ej. 'AAPL_2025_FY').
- **ticker** (TEXT, FOREIGN KEY)
- **year** (INTEGER)
- **period** (TEXT): 'FY' (Full Year) or 'Q1', 'Q2', etc.
- *Partidas del Income Statement:* **revenue**, **gross_profit**, **ebit**, **net_income**, **eps**.

- *Partidas del Balance Sheet:* `total_assets`, `total_debt`, `cash_and_equivalents`, `working_capital`, `total_shareholders_equity`.
- *Partidas del Cash Flow:* `operating_cash_flow`, `capex`, `free_cash_flow`.

Tabla 3: `fact_market_data` (Precios y Métricas de Mercado Dinámicas)

- `ticker` (TEXT, FOREIGN KEY)
- `date` (DATE)
- `close_price` (NUMERIC)
- `market_cap` (NUMERIC)
- `enterprise_value` (NUMERIC)
- PRIMARY KEY (`ticker`, `date`)

Filtro Automático de Crecimiento (*High-Growth Flag*). **Lógica:** Si Revenue CAGR (3Y)>15%, el flag `is_high_growth` se activa como `True`.

Cobertura de Deuda con FCF. **Lógica:** Debe ser <2.5. Si el FCF es negativo o la métrica es superior a 2.5, el ticker se descarta automáticamente del portfolio estratégico perpetuo.

$$\text{Años de Cobertura} = \frac{\text{Total Debt} - \text{Cash}}{\text{Free Cash Flow}}$$

Si en un año el resultado es negativo (créditos fiscales), el código lo limitará automáticamente a cero para no distorsionar el ROIC.

$$\text{Effective Tax Rate} = \frac{\text{Tax Provision}}{\text{Pretax Income}}$$

$$\text{ROIC} = \frac{\text{EBIT} \times (1 - \text{Tax Rate})}{\text{Total Debt} + \text{Total Equity} - \text{Cash}}$$

La mayoría de webs dan una Beta fija (normalmente a 3 o 5 años). Esto es impreciso porque el riesgo de una empresa cambia si se endeuda más o menos:

Beta Dinámica (Market Beta): Tu código calculará la covarianza de los rendimientos diarios de la acción frente al S&P 500 durante los últimos 36 meses de forma histórica.

1. **Beta Unlevered (Beta del Negocio Puro):** Elimina el riesgo financiero de la deuda. Mide el riesgo del negocio como si no tuviera un solo dólar de deuda.
2. **Beta Levered Ajustada:** Introduce la estructura de capital exacta de la empresa para calcular el coste de capital real que le corresponde hoy.

$$\beta_{\text{Unlevered}} = \frac{\beta_{\text{Levered}}}{1 + (1 - \text{Tax Rate}) \times \left(\frac{\text{Total Debt}}{\text{Market Cap}} \right)}$$

$$\beta_{\text{Levered New}} = \beta_{\text{Unlevered}} \times \left[1 + (1 - \text{Tax Rate}) \times \left(\frac{\text{Total Debt}}{\text{Market Cap}} \right) \right]$$

El dilema del ROIC vs. WACC condicional. Usaremos la tasa impositiva efectiva basada en el Income Statement.

$$\text{Revenue CAGR (3Y)} = \left(\frac{\text{Revenue}_t}{\text{Revenue}_{t-3}} \right)^{\frac{1}{3}} - 1$$

Screeners_Fundamental_Perpetuo/

|

— data/

← Carpeta vacía (aquí DuckDB guardará la base de datos)

— config.py	← Aquí se guardarán las reglas (umbrales de ROIC, industrias...)
— database_manager.py	← EL PRIMER MÓDULO: Creación y conexión a SQL
— data_loader.py	← El módulo ETL (descarga de datos)
— analytics_engine.py	← El motor de cálculo matemático
— main.ipynb	← Cuaderno Jupyter para ejecutar todo y ver los resultados

```

[ Yahoo Finance API ]
    |
    ▼ (Descarga via yfinance en formato Pandas DataFrame)
[ Pandas DataFrames ] --> Balance Sheet, Income Statement, Financials
    |
    ▼ (Transformación: Transponer filas/columnas, mapear industria)
[ Clean DataFrames ]
    |
    ▼ (Carga eficiente usando DuckDB en lotes)
[ Tu Archivo .db (DuckDB) ]

```